Numerical Analysis

Chapter 7: Initial Value Problems for Ordinary Differential Equations

Department of Mathematics & Statistics
Missouri University of Science & Technology

Textbook: Classical and Modern Numerical Analysis: Theory, Methods and Practice
A. S. Ackleh, E. J. Allen, R. B. Kearfott and P. Seshaiyer
CRC Press, Boca Raton, FL, 2010

Outline

1. Introduction
2. Euler methods
3. Runge-Kutta methods
4. Analysis
5. Other topics

Outline

1 Introduction

2 Euler methods

3 Runge-Kutta methods

4 Analysis

5 Other topics

Initial value problem

- Initial value problem (IVP) for a first order ordinary differential equation (ODE):

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ y(a) = g_a$$

given the initial value g_a.

Initial value problem

- Initial value problem (IVP) for a first order ordinary differential equation (ODE):

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ y(a) = g_a$$

given the initial value g_a.

Theorem (I)

Let $f(t, y)$ be a continuous function defined on

$$S = \{(t, y) : t, y \in \mathbb{R}, |t - a| < \gamma, |y| < \infty\}.$$

If f satisfies a Lipschitz condition in the y variable over S. Then $y'(t) = f(t, y(t))$ with $y(a) = g_a$ has a unique solution for $|t - a| < \gamma$.

Outline

1 Introduction

2 Euler methods

3 Runge-Kutta methods

4 Analysis

5 Other topics

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.

- How to form the mesh and construct the numerical schemes?

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.
- How to form the mesh and construct the numerical schemes?
- Stability

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.
- How to form the mesh and construct the numerical schemes?
- Stability
- Consistency

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.
- How to form the mesh and construct the numerical schemes?
- Stability
- Consistency
- Convergence

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.
- How to form the mesh and construct the numerical schemes?
- Stability
- Consistency
- Convergence
- Accuracy

What's the target problem?

- Approximate the solutions of initial value problems for ordinary differential equations by using numerical differentiations in Chapter 6.
- How to form the mesh and construct the numerical schemes?
- Stability
- Consistency
- Convergence
- Accuracy
- .....

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ y(a) = g_a$$

given the initial value g_a.

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a$$

given the initial value g_a.

- Assume that we have a uniform partition of $[a, b]$ into J elements with mesh size h.

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a$$

given the initial value g_a.

- Assume that we have a uniform partition of $[a, b]$ into J elements with mesh size h.

- The mesh nodes are $t_j = a + jh, \ \ j = 0, 1, \cdots, J$.

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a$$

given the initial value g_a.

- Assume that we have a uniform partition of $[a, b]$ into J elements with mesh size h.

- The mesh nodes are $t_j = a + jh, \ \ j = 0, 1, \cdots, J$.

- The goal of finite difference method: Discretize the equation and obtain the numerical solution at all of the mesh nodes.

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a$$

given the initial value g_a.

- Assume that we have a uniform partition of $[a, b]$ into J elements with mesh size h.
- The mesh nodes are $t_j = a + jh, \ \ j = 0, 1, \cdots, J$.
- The goal of finite difference method: Discretize the equation and obtain the numerical solution at all of the mesh nodes.
- Assume y_j is the numerical solution of $y(t_j)$.

Basic idea

- Consider the IVP

$$y'(t) = f(t, y(t)) \ (a \le t \le b), \ \ y(a) = g_a$$

given the initial value g_a.

- Assume that we have a uniform partition of $[a, b]$ into J elements with mesh size h.
- The mesh nodes are $t_j = a + jh, \ \ j = 0, 1, \cdots, J$.
- The goal of finite difference method: Discretize the equation and obtain the numerical solution at all of the mesh nodes.
- Assume y_j is the numerical solution of $y(t_j)$.
- Then the initial condition implies: $y_0 = y(a) = g_a$.

Basic idea

- A straightforward discretization of $f(t, y(t))$ at t_j is $f(t_j, y_j)$.

Basic idea

- A straightforward discretization of $f(t, y(t))$ at t_j is $f(t_j, y_j)$.
- How about the discretization of $y'(t)$ at t_j?

Basic idea

- A straightforward discretization of $f(t, y(t))$ at t_j is $f(t_j, y_j)$.
- How about the discretization of $y'(t)$ at t_j?
- We derive the following conclusions by using Taylor's expansion in Chapter 6:

$$y'(t_j) = \frac{y(t_{j+1}) - y(t_j)}{h} + O(h),$$

Basic idea

- A straightforward discretization of $f(t, y(t))$ at t_j is $f(t_j, y_j)$.
- How about the discretization of $y'(t)$ at t_j?
- We derive the following conclusions by using Taylor's expansion in Chapter 6:

$$
\begin{aligned}
y'(t_j) &= \frac{y(t_{j+1}) - y(t_j)}{h} + O(h), \\
y'(t_j) &= \frac{y(t_j) - y(t_{j-1})}{h} + O(h),
\end{aligned}
$$

Basic idea

- A straightforward discretization of $f(t, y(t))$ at t_j is $f(t_j, y_j)$.
- How about the discretization of $y'(t)$ at t_j?
- We derive the following conclusions by using Taylor's expansion in Chapter 6:

$$
\begin{aligned}
y'(t_j) &= \frac{y(t_{j+1}) - y(t_j)}{h} + O(h), \\
y'(t_j) &= \frac{y(t_j) - y(t_{j-1})}{h} + O(h), \\
y'(t_j) &= \frac{y(t_{j+1}) - y(t_{j-1})}{2h} + O(h^2).
\end{aligned}
$$

Basic idea

Hence we obtain the following difference schemes:

Basic idea

Hence we obtain the following difference schemes:

- Forward difference for $y'(t_j) \approx \frac{y_{j+1}-y_j}{h}$.

Basic idea

Hence we obtain the following difference schemes:

- Forward difference for $y'(t_j) \approx \frac{y_{j+1}-y_j}{h}$.
- Backward difference for $y'(t_j) \approx \frac{y_j-y_{j-1}}{h}$.

Basic idea

Hence we obtain the following difference schemes:

- Forward difference for $y'(t_j) \approx \frac{y_{j+1}-y_j}{h}$.
- Backward difference for $y'(t_j) \approx \frac{y_j-y_{j-1}}{h}$.
- Centered difference for $y'(t_j) \approx \frac{y_{j+1}-y_{j-1}}{2h}$.

Forward Euler method

- Forward Euler scheme:

$$y'(t) = f(t, y(t))$$

Forward Euler method

- Forward Euler scheme:

$$\begin{aligned} & y'(t) = f(t, y(t)) \\ \Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \end{aligned}$$

Forward Euler method

- Forward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & \frac{y(t_{j+1}) - y(t_j)}{h} + O(h) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1
\end{aligned}
$$

Forward Euler method

- Forward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & \frac{y(t_{j+1}) - y(t_j)}{h} + O(h) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & \frac{y_{j+1} - y_j}{h} = f(t_j, y_j),\ j = 0, \cdots, J-1
\end{aligned}
$$

Forward Euler method

- Forward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & \frac{y(t_{j+1}) - y(t_j)}{h} + O(h) = f(t_j, y(t_j)),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & \frac{y_{j+1} - y_j}{h} = f(t_j, y_j),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & y_{j+1} = y_j + h \cdot f(t_j, y_j),\ j = 0, \cdots, J-1, \\
& y_0 = y(a) = g_a.
\end{aligned}
$$

Practice in Matlab for forward Euler method

- Example 1: Consider the ODE

$$y'(t) = t + y \ (a \leq t \leq b), \ y(a) = 2.$$

Practice in Matlab for forward Euler method

- Example 1: Consider the ODE

$$y'(t) = t + y \ (a \leq t \leq b), \ y(a) = 2.$$

- Then by the forward Euler method:

$$y_0 = 2, \ y_{j+1} = y_j + h \ (t_j + y_j), \ j = 0, 1, \cdots, J-1.$$

Practice in Matlab for forward Euler method

- Example 1: Consider the ODE

$$y'(t) = t + y \ (a \leq t \leq b), \ y(a) = 2.$$

- Then by the forward Euler method:

$$y_0 = 2, \ y_{j+1} = y_j + h \ (t_j + y_j), \ j = 0, 1, \cdots, J-1.$$

- Let's code for this simple algorithm together!
- Open your Matlab!

Practice in Matlab for forward Euler method

- Choose $a = 1$, $b = 1.5$, $h = 0.1$. Then $J = 5$. Numerical results from the code:

j	t_j	y_j	$y(t_j)$
0	1	2.0000	2.0000
1	1.1	2.3000	2.3207
2	1.2	2.6400	2.6856
3	1.3	3.0240	3.0994
4	1.4	3.4564	3.5673
5	1.5	3.9420	4.0948

Backward Euler method

- Backward Euler scheme:

$$y'(t) = f(t, y(t))$$

Backward Euler method

- Backward Euler scheme:

$$\begin{aligned} & y'(t) = f(t, y(t)) \\ \Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \end{aligned}$$

Backward Euler method

- Backward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y(t_j) - y(t_{j-1})}{h} + O(h) = f(t_j, y(t_j)),\ j = 1, \cdots, J
\end{aligned}
$$

Backward Euler method

- Backward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y(t_j) - y(t_{j-1})}{h} + O(h) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_j - y_{j-1}}{h} = f(t_j, y_j),\ j = 1, \cdots, J
\end{aligned}
$$

Backward Euler method

- Backward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y(t_j) - y(t_{j-1})}{h} + O(h) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_j - y_{j-1}}{h} = f(t_j, y_j),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_{j+1} - y_j}{h} = f(t_{j+1}, y_{j+1}),\ j = 0, \cdots, J-1
\end{aligned}
$$

Backward Euler method

- Backward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y(t_j) - y(t_{j-1})}{h} + O(h) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_j - y_{j-1}}{h} = f(t_j, y_j),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_{j+1} - y_j}{h} = f(t_{j+1}, y_{j+1}),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & y_{j+1} = y_j + h \cdot f(t_{j+1}, y_{j+1}),\ j = 0, \cdots, J-1, \\
& y_0 = y(a).
\end{aligned}
$$

Backward Euler method

- Backward Euler scheme:

$$
\begin{aligned}
& y'(t) = f(t, y(t)) \\
\Rightarrow \quad & y'(t_j) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y(t_j) - y(t_{j-1})}{h} + O(h) = f(t_j, y(t_j)),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_j - y_{j-1}}{h} = f(t_j, y_j),\ j = 1, \cdots, J \\
\Rightarrow \quad & \frac{y_{j+1} - y_j}{h} = f(t_{j+1}, y_{j+1}),\ j = 0, \cdots, J-1 \\
\Rightarrow \quad & y_{j+1} = y_j + h \cdot f(t_{j+1}, y_{j+1}),\ j = 0, \cdots, J-1, \\
& y_0 = y(a).
\end{aligned}
$$

- How to solve this equation for y_{j+1}, especially when $f(t, y)$ is a nonlinear function in y? Chapter 2!

Outline

1 Introduction

2 Euler methods

3 Runge-Kutta methods

4 Analysis

5 Other topics

Runge-Kutta methods

- Given the initial value g_a, consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a.$$

Runge-Kutta methods

- Given the initial value g_a, consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ y(a) = g_a.$$

- General formulation of Rth order Runge-Kutta method:

$$y_{j+1} = y_j + h \sum_{r=1}^{R} c_r K_r,$$

$$y_0 = y(a),$$

Runge-Kutta methods

- Given the initial value g_a, consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ y(a) = g_a.$$

- General formulation of Rth order Runge-Kutta method:

$$y_{j+1} = y_j + h \sum_{r=1}^{R} c_r K_r,$$

$$y_0 = y(a),$$

where

$$K_1 = f(t_j, y_j),$$

Runge-Kutta methods

- Given the initial value g_a, consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a.$$

- General formulation of Rth order Runge-Kutta method:

$$y_{j+1} = y_j + h \sum_{r=1}^{R} c_r K_r,$$

$$y_0 = y(a),$$

where

$$K_1 = f(t_j, y_j),$$

$$K_r = f(t_j + a_r h, y_j + h \sum_{s=1}^{r-1} b_{rs} K_s), \ \ r = 2, \cdots, R,$$

Runge-Kutta methods

- Given the initial value g_a, consider the IVP

$$y'(t) = f(t, y(t)) \ (a \leq t \leq b), \ \ y(a) = g_a.$$

- General formulation of Rth order Runge-Kutta method:

$$y_{j+1} = y_j + h \sum_{r=1}^{R} c_r K_r,$$

$$y_0 = y(a),$$

where

$$K_1 = f(t_j, y_j),$$

$$K_r = f(t_j + a_r h, y_j + h \sum_{s=1}^{r-1} b_{rs} K_s), \ \ r = 2, \cdots, R,$$

$$a_r = \sum_{s=1}^{r-1} b_{rs}.$$

Runge-Kutta methods

Remark

For different Runge-Kutta methods of different orders, different b_{rs}, a_r, and c_r can be obtained through the regular derivation based on Taylor's expansion. See the examples below for more details.

Runge-Kutta methods

- 2nd order Runge-Kutta method: $b_{21} = 1$, $a_2 = 1$, and $c_1 = c_2 = \frac{1}{2}$.

Runge-Kutta methods

- 2nd order Runge-Kutta method: $b_{21} = 1$, $a_2 = 1$, and $c_1 = c_2 = \frac{1}{2}$.
- Hence the scheme is

$$y_{j+1} = y_j + \frac{1}{2}h(K_1 + K_2),$$
$$y_0 = y(a) = g_a,$$

Runge-Kutta methods

- 2nd order Runge-Kutta method: $b_{21} = 1$, $a_2 = 1$, and $c_1 = c_2 = \frac{1}{2}$.
- Hence the scheme is

$$y_{j+1} = y_j + \frac{1}{2}h(K_1 + K_2),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$

Runge-Kutta methods

- 2nd order Runge-Kutta method: $b_{21} = 1$, $a_2 = 1$, and $c_1 = c_2 = \frac{1}{2}$.
- Hence the scheme is

$$y_{j+1} = y_j + \frac{1}{2}h(K_1 + K_2),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$
$$K_2 = f(t_j + h, y_j + hK_1).$$

Runge-Kutta methods

- 4th order Runge-Kutta method:

$$y_{j+1} = y_j + \frac{1}{6}h(K_1 + 2K_2 + 2K_3 + K_4),$$
$$y_0 = y(a) = g_a,$$

Runge-Kutta methods

- 4th order Runge-Kutta method:

$$y_{j+1} = y_j + \frac{1}{6}h(K_1 + 2K_2 + 2K_3 + K_4),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$

Runge-Kutta methods

- 4th order Runge-Kutta method:

$$y_{j+1} = y_j + \frac{1}{6}h(K_1 + 2K_2 + 2K_3 + K_4),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$
$$K_2 = f(t_j + \frac{1}{2}h, y_j + \frac{1}{2}hK_1),$$

Runge-Kutta methods

- 4th order Runge-Kutta method:

$$y_{j+1} = y_j + \frac{1}{6}h(K_1 + 2K_2 + 2K_3 + K_4),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$
$$K_2 = f(t_j + \frac{1}{2}h, y_j + \frac{1}{2}hK_1),$$
$$K_3 = f(t_j + \frac{1}{2}h, y_j + \frac{1}{2}hK_2),$$

Runge-Kutta methods

- 4th order Runge-Kutta method:

$$y_{j+1} = y_j + \frac{1}{6}h(K_1 + 2K_2 + 2K_3 + K_4),$$
$$y_0 = y(a) = g_a,$$

where

$$K_1 = f(t_j, y_j),$$
$$K_2 = f(t_j + \frac{1}{2}h, y_j + \frac{1}{2}hK_1),$$
$$K_3 = f(t_j + \frac{1}{2}h, y_j + \frac{1}{2}hK_2),$$
$$K_4 = f(t_j + h, y_j + hK_3).$$

Outline

1 Introduction

2 Euler methods

3 Runge-Kutta methods

4 Analysis

5 Other topics

Truncation error and consistency

Definition (Local truncation error)

The local truncation error is the error introduced in a single iteration of the method, assuming that everything in the previous steps is perfectly accurate.

Truncation error and consistency

Definition (Local truncation error)

The local truncation error is the error introduced in a single iteration of the method, assuming that everything in the previous steps is perfectly accurate.

Definition (Global truncation error)

The global truncation error is the accumulation of the local truncation error over all of the iterations, assuming that everything in the initial step is perfectly accurate.

Truncation error and consistency

Definition (Local truncation error)

The local truncation error is the error introduced in a single iteration of the method, assuming that everything in the previous steps is perfectly accurate.

Definition (Global truncation error)

The global truncation error is the accumulation of the local truncation error over all of the iterations, assuming that everything in the initial step is perfectly accurate.

Definition (Consistency)

We say that a method is consistent with the differential equation with initial condition if the local truncation error approaches zero as $h \to 0$.

Truncation error and consistency for forward Euler method

- Assume that everything in the previous steps is perfectly accurate. That is, $y_j = y(t_j)$ when we consider y_{j+1} in the forward Euler method

$$y_{j+1} = y_j + hf(t_j, y_j).$$

Truncation error and consistency for forward Euler method

- Assume that everything in the previous steps is perfectly accurate. That is, $y_j = y(t_j)$ when we consider y_{j+1} in the forward Euler method

$$y_{j+1} = y_j + hf(t_j, y_j).$$

- Since the original ODE is $y'(t) = f(t, y(t))$, then

$$y'(t_j) = f(t_j, y(t_j)).$$

Truncation error and consistency for forward Euler method

- Assume that everything in the previous steps is perfectly accurate. That is, $y_j = y(t_j)$ when we consider y_{j+1} in the forward Euler method

$$y_{j+1} = y_j + hf(t_j, y_j).$$

- Since the original ODE is $y'(t) = f(t, y(t))$, then

$$y'(t_j) = f(t_j, y(t_j)).$$

- Hence

$$\begin{aligned} y_{j+1} &= y(t_j) + hf(t_j, y(t_j)) \\ &= y(t_j) + hy'(t_j). \end{aligned}$$

Truncation error and consistency for forward Euler method

- On the other hand, Taylor expansion of $y(t_{j+1})$ at t_j provides

$$\begin{aligned} y(t_{j+1}) &= y(t_j + h) \\ &= y(t_j) + hy'(t_j) + \frac{h^2}{2}y''(t_j) + O(h^3). \end{aligned}$$

Truncation error and consistency for forward Euler method

- On the other hand, Taylor expansion of $y(t_{j+1})$ at t_j provides

$$\begin{aligned} y(t_{j+1}) &= y(t_j + h) \\ &= y(t_j) + hy'(t_j) + \frac{h^2}{2}y''(t_j) + O(h^3). \end{aligned}$$

- Then the local truncation errors of the forward Euler method is

$$y(t_{j+1}) - y_{j+1} = \frac{h^2}{2}y''(t_j) + O(h^3) = O(h^2), \text{ as } h \to 0.$$

Truncation error and consistency for forward Euler method

- On the other hand, Taylor expansion of $y(t_{j+1})$ at t_j provides

$$\begin{aligned} y(t_{j+1}) &= y(t_j + h) \\ &= y(t_j) + hy'(t_j) + \frac{h^2}{2}y''(t_j) + O(h^3). \end{aligned}$$

- Then the local truncation errors of the forward Euler method is

$$y(t_{j+1}) - y_{j+1} = \frac{h^2}{2}y''(t_j) + O(h^3) = O(h^2), \text{ as } h \to 0.$$

- Therefore, the local truncation of the forward Euler method is $O(h^2)$.

Truncation error and consistency for forward Euler method

- On the other hand, Taylor expansion of $y(t_{j+1})$ at t_j provides

$$\begin{aligned} y(t_{j+1}) &= y(t_j + h) \\ &= y(t_j) + hy'(t_j) + \frac{h^2}{2}y''(t_j) + O(h^3). \end{aligned}$$

- Then the local truncation errors of the forward Euler method is

$$y(t_{j+1}) - y_{j+1} = \frac{h^2}{2}y''(t_j) + O(h^3) = O(h^2), \text{ as } h \to 0.$$

- Therefore, the local truncation of the forward Euler method is $O(h^2)$.

- Since $O(h^2)$ approaches zero as $h \to 0$, then forward Euler method is consistent.

Stability

Definition (Zero stability)

A difference scheme for which perturbations remain bounded in the limit of $h \to 0$ is said to be zero-stable.

Stability

Definition (Zero stability)

A difference scheme for which perturbations remain bounded in the limit of $h \to 0$ is said to be zero-stable.

Theorem

All the one-step methods are zero-stable.

Stability

Definition (Zero stability)

A difference scheme for which perturbations remain bounded in the limit of $h \to 0$ is said to be zero-stable.

Theorem

All the one-step methods are zero-stable.

Definition (Absolute stability)

A numerical method is called absolute stable (A-stable), if when it applied to

$$y' = \lambda y, \text{ with } \quad \mathrm{Re}(\lambda) < 0, \quad y(0) = g_0,$$

the numerical solution $y_j \to 0$ as $j \to \infty$, for any step size h.

Stability for forward Euler method

- For absolute stability, we consider the ODE

$$y'(t) = \lambda y, \ y(0) = g_0.$$

Stability for forward Euler method

- For absolute stability, we consider the ODE

$$y'(t) = \lambda y, \ y(0) = g_0.$$

- Then the forward Euler scheme becomes

$$\frac{y_{j+1} - y_j}{h} = \lambda y_j, \quad y_0 = y(0) = g_0.$$

Stability for forward Euler method

- For absolute stability, we consider the ODE

$$y'(t) = \lambda y, \ y(0) = g_0.$$

- Then the forward Euler scheme becomes

$$\frac{y_{j+1} - y_j}{h} = \lambda y_j, \quad y_0 = y(0) = g_0.$$

- Hence

$$\begin{aligned} y_{j+1} &= (1 + h\lambda) y_j \\ &= (1 + h\lambda)^2 y_{j-1} \\ &= \cdots \\ &= (1 + h\lambda)^{j+1} y_0 \\ &= g_0 (1 + h\lambda)^{j+1}. \end{aligned}$$

Stability for forward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need

$$|1 + h\lambda| < 1.$$

Stability for forward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need

$$|1 + h\lambda| < 1.$$

- For real λ, it is equivalent to

$$-1 < 1 + h\lambda < 1.$$

Stability for forward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need

$$|1 + h\lambda| < 1.$$

- For real λ, it is equivalent to

$$-1 < 1 + h\lambda < 1.$$

- Hence

$$0 < h < \frac{2}{\lambda}.$$

Stability for forward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need

$$|1 + h\lambda| < 1.$$

- For real λ, it is equivalent to

$$-1 < 1 + h\lambda < 1.$$

- Hence

$$0 < h < \frac{2}{\lambda}.$$

- Conclusion: Forward Euler method is a conditionally stable method, if $h < 2/|\lambda|$ holds.

Stability for backward Euler method

- For absolute stability, we again consider the ODE $y'(t) = \lambda y,\ y(0) = g_0$.

Stability for backward Euler method

- For absolute stability, we again consider the ODE $y'(t) = \lambda y,\ y(0) = g_0$.
- Then the backward Euler scheme becomes

$$\frac{y_{j+1} - y_j}{h} = \lambda y_{j+1}.$$

Stability for backward Euler method

- For absolute stability, we again consider the ODE $y'(t) = \lambda y,\ y(0) = g_0$.
- Then the backward Euler scheme becomes

$$\frac{y_{j+1} - y_j}{h} = \lambda y_{j+1}.$$

- Hence

$$\begin{aligned} y_{j+1} &= (\frac{1}{1-h\lambda})y_j \\ &= (\frac{1}{1-h\lambda})^2 y_{j-1} \\ &= \cdots \\ &= (\frac{1}{1-h\lambda})^{j+1} y_0 \\ &= g_0(\frac{1}{1-h\lambda})^{j+1}. \end{aligned}$$

Stability for backward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need to have

$$|1 - h\lambda| > 1.$$

Stability for backward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need to have

$$|1 - h\lambda| > 1.$$

- For real λ, it is equivalent to

$$1 - h\lambda > 1 \quad \text{or} \quad 1 - h\lambda < -1.$$

Stability for backward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need to have

$$|1 - h\lambda| > 1.$$

- For real λ, it is equivalent to

$$1 - h\lambda > 1 \quad \text{or} \quad 1 - h\lambda < -1.$$

- Since $\lambda < 0$, we have

$$1 - h\lambda > 1 \quad \text{for any} \quad h > 0.$$

Stability for backward Euler method

- In order to make $y_j \to 0$ as $j \to \infty$, we need to have

$$|1 - h\lambda| > 1.$$

- For real λ, it is equivalent to

$$1 - h\lambda > 1 \quad \text{or} \quad 1 - h\lambda < -1.$$

- Since $\lambda < 0$, we have

$$1 - h\lambda > 1 \quad \text{for any} \quad h > 0.$$

- Conclusion: Backward Euler method is A-stable.

Convergence

Definition (Convergence)

A numerical method is said to be convergent if the numerical solution approaches the exact solution as the step size $h \to 0$.

Convergence

Definition (Convergence)

A numerical method is said to be convergent if the numerical solution approaches the exact solution as the step size $h \to 0$.

Theorem (Dahlquist Theorem)

Consistency + *Zero-stability* $\Leftrightarrow$ *Convergence*

Convergence

Definition (Convergence)

A numerical method is said to be convergent if the numerical solution approaches the exact solution as the step size $h \to 0$.

Theorem (Dahlquist Theorem)

Consistency + *Zero-stability* $\Leftrightarrow$ *Convergence*

Theorem

All consistent one-step methods are convergent.

Outline

1 Introduction

2 Euler methods

3 Runge-Kutta methods

4 Analysis

5 Other topics

More schemes

See the midterm project for more details of the following three schemes:

- Trapezoidal scheme(Crank-Nicolson scheme if it's applied to PDE):

$$\frac{y_{j+1} - y_j}{h} = \frac{f(t_{j+1}, y_{j+1}) + f(t_j, y_j)}{2};$$

More schemes

See the midterm project for more details of the following three schemes:

- Trapezoidal scheme(Crank-Nicolson scheme if it's applied to PDE):

$$\frac{y_{j+1} - y_j}{h} = \frac{f(t_{j+1}, y_{j+1}) + f(t_j, y_j)}{2};$$

- Two-step backward differentiation:

$$\frac{3y_{j+1} - 4y_j + y_{j-1}}{2h} = f(t_{j+1}, y_{j+1});$$

More schemes

See the midterm project for more details of the following three schemes:

- Trapezoidal scheme(Crank-Nicolson scheme if it's applied to PDE):

$$\frac{y_{j+1} - y_j}{h} = \frac{f(t_{j+1}, y_{j+1}) + f(t_j, y_j)}{2};$$

- Two-step backward differentiation:

$$\frac{3y_{j+1} - 4y_j + y_{j-1}}{2h} = f(t_{j+1}, y_{j+1});$$

- Three-step backward differentiation:

$$\frac{11y_{j+1} - 18y_j + 9y_{j-1} - 2y_{j-2}}{6h} = f(t_{j+1}, y_{j+1}).$$

Topics: Independent study problems

- Runge-Kutta methods and other single step methods (section 7.3 and 7.4)
- Multi-step methods (section 7.5)
- Predictor-Corrector methods (section 7.6)
- Stiff systems (section 7.7)
- Extrapolation methods (section 7.8)
- Higher order ODEs
- System of ODEs
- Stability analysis
- Consistency analysis
- Convergence analysis
- .....